



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Ingegneria del Software: Tecniche Avanzate

CodeSmile



Change Request Document
Versione 3.0

Team

Choaib Goumri NF22500223
Mattia Gallucci NF22500127

Repository GitHub

Anno Accademico 2025–2026

Indice

1	Introduzione	1
1.1	Contesto e Problematica	1
1.2	Architettura del Sistema	1
1.2.1	Core Backend (Analisi AST)	1
1.2.2	Web Infrastructure (Microservizi)	2
2	Change Requests	3
2.1	CR1: Stabilità e Accessibilità dell'Ambiente (Docker)	3
2.1.1	Descrizione	3
2.1.2	Il Problema	3
2.1.3	Soluzione proposta	3
2.2	CR2: Feedback Visivo di Avanzamento (Progress Bar)	4
2.2.1	Descrizione	4
2.2.2	Il Problema	4
2.2.3	Soluzione proposta	4
2.3	CR3: Rilevamento Code Smell: Randomness Uncontrolled	4
2.3.1	Descrizione	4
2.3.2	Il Problema	4
2.3.3	Soluzione proposta	4
2.4	CR4: Rilevamento Code Smell: Hyperparameter not explicitly set	5
2.4.1	Descrizione	5
2.4.2	Il Problema	5
2.4.3	Soluzione proposta	5
2.5	CR5: Nuova Metrica di "Densità degli Smell"	5
2.5.1	Descrizione	5
2.5.2	Il Problema	5
2.5.3	Soluzione proposta	6
2.6	Riepilogo Stato Modifiche	6

1 Introduzione

1.1 Contesto e Problematica

L'avvento dei *ML-enabled systems* ha introdotto nuove sfide di manutenzione rispetto al software tradizionale. La pressione sul *time-to-market* favorisce l'accumulo di **AI-Specific Technical Debt**, compromettendo stabilità, riproducibilità ed evoluzione. Questo debito si manifesta attraverso i **Machine Learning Specific Code Smells (ML-CSs)**, difetti architetturali o implementativi legati all'uso improprio di librerie (Pandas, PyTorch, TensorFlow).

Soluzione Proposta

CodeSmile è un tool di analisi statica per progetti Python che mira a ridurre il debito tecnico identificando 16 tipologie di smell in quattro aree critiche: *Data Debt*, *Model Debt*, *Configuration Debt* ed *Ethics Debt*.

1.2 Architettura del Sistema

Il sistema adotta un design modulare orientato agli oggetti e si divide in due macro-ambiti: il core (Python Package) e l'infrastruttura Web (Microservizi).

1.2.1 Core Backend (Analisi AST)

Basato sull'analisi degli *Abstract Syntax Trees* (AST), è strutturato nei seguenti package:

- **cli:** Gestisce l'interfaccia a riga di comando e l'orchestrazione (`cli_runner`).
- **code_extractor:** Estrae semantica da DataFrames, librerie, modelli e variabili.
- **components:** Cuore operativo. Include l'`Inspector` (analisi file), il `Project Analyzer` (gestione progetto) e il `Rule Checker`.
- **detection_rules:** Contiene la gerarchia delle regole (`Smell`), divise in *API-Specific* (es. errori Pandas) e *Generic*.
- **gui & report:** Gestione interfaccia desktop (Tkinter) e generazione output (CSV/Excel).

1.2.2 Web Infrastructure (Microservizi)

Architettura distribuita con Frontend in React/Next.js e Backend Python basato su FastAPI:

- **API Gateway:** Punto di ingresso unico, gestisce routing e sicurezza.
- **AI Analysis Service:** Rilevamento semantico tramite LLM.
- **Static Analysis Service:** Esegue l'analisi AST riutilizzando la logica core dell'Inspector.
- **Report Service:** Aggrega i dati e genera statistiche visualizzabili.

2 Change Requests

Questo capitolo descrive le richieste di modifica (CR) identificate per l'evoluzione del sistema CodeSmile. Le richieste coprono miglioramenti infrastrutturali, ottimizzazioni dell'esperienza utente ed estensioni delle capacità di analisi del software.

2.1 CR1: Stabilità e Accessibilità dell'Ambiente (Docker)

Tipologia di manutenzione: Correttiva.

2.1.1 Descrizione

L'obiettivo di questa richiesta è garantire che l'ambiente di sviluppo e distribuzione (basato su container) sia stabile, facilmente installabile e accessibile su qualsiasi macchina ospite senza configurazioni manuali complesse. In particolare, si intende automatizzare la configurazione tra locale e Docker, eliminando la necessità di modificare manualmente import e URL nei servizi.

2.1.2 Il Problema

Attualmente, l'avvio del sistema può presentare problemi di comunicazione tra i vari servizi (Web App e API) e la macchina dell'utente. Il problema critico risiede nella gestione delle dipendenze e degli import Python: il progetto fa una distinzione rigida tra esecuzione locale e con Docker. Per eseguire l'applicazione in Docker, l'utente è costretto a commentare manualmente le righe di codice relative all'esecuzione locale e decommentare quelle relative a Docker (e viceversa). Questo approccio manuale è propenso ad errori e peggiora l'esperienza utente.

2.1.3 Soluzione proposta

Si propone di standardizzare la configurazione e rendere il codice agnostico rispetto all'ambiente di esecuzione. La soluzione implementata prevede:

1. Import Dinamici: Utilizzo di blocchi 'try-except' o logica condizionale nei file sorgente Python ('main.py', 'routers', ecc.) per rilevare automaticamente se il modulo deve essere importato con il path relativo (Docker) o assoluto/modificato (Locale), senza intervento umano.
2. Configurazione via Variabili d'Ambiente: Gestione degli URL dei microservizi nel Gateway tramite variabili d'ambiente (con valori di default per 'localhost'), sovrascritte automaticamente dal file 'docker-compose.yml' per l'ambiente containerizzato.

2.2 CR2: Feedback Visivo di Avanzamento (Progress Bar)

Tipologia di manutenzione: Perfettiva.

2.2.1 Descrizione

Questa richiesta mira a introdurre una barra di avanzamento nell'interfaccia grafica (GUI) per comunicare all'utente lo stato dell'analisi in tempo reale, migliorando l'usabilità del sistema.

2.2.2 Il Problema

Durante l'analisi di progetti di grandi dimensioni, l'interfaccia attuale non fornisce un feedback chiaro sulla percentuale di completamento. L'utente vede solo dei log testuali o, in alcuni casi, l'interfaccia può sembrare "bloccata", creando incertezza sul fatto che il programma stia effettivamente lavorando.

2.2.3 Soluzione proposta

Verrà integrato un indicatore visivo (barra di progresso) che mostra chiaramente quanto manca alla fine dell'operazione. Il sistema calcolerà il totale dei file da esaminare e aggiornerà la barra passo dopo passo, migliorando la percezione delle prestazioni e la trasparenza del processo.

2.3 CR3: Rilevamento Code Smell: Randomness Uncontrolled

Tipologia di manutenzione: Evolutiva.

2.3.1 Descrizione

Si richiede l'implementazione di una nuova regola di rilevamento per identificare la mancata gestione della casualità negli algoritmi di Machine Learning.

2.3.2 Il Problema

Molti algoritmi di Machine Learning utilizzano numeri casuali per la suddivisione del dataset, l'inizializzazione dei centroidi negli algoritmi di clustering. Se questa casualità non viene controllata fissando un **seed**, ogni esecuzione del codice produce risultati diversi. Questo rende impossibile la riproducibilità scientifica degli esperimenti: non si può sapere se un miglioramento delle performance sia dovuto al codice o alla fortuna.

2.3.3 Soluzione proposta

Il sistema utilizzerà l'analisi Abstract Syntax Tree per identificare istanziazioni di modelli ML e verificare la presenza del parametro di controllo della casualità, in caso di assenza il sistema segnalerà lo smell.

2.4 CR4: Rilevamento Code Smell: Hyperparameter not explicitly set

Tipologia di manutenzione: Perfettiva.

2.4.1 Descrizione

Si richiede l'aggiornamento della regola di rilevamento per "Hyperparameter not explicitly set". Attualmente la regola segnala solo i casi in cui il costruttore del modello è completamente privo di argomenti. La nuova implementazione dovrà essere più granulare: il tool dovrà rilevare l'assenza di specifici iperparametri critici predefiniti per ogni modello, anche qualora siano stati specificati altri parametri secondari.

2.4.2 Il Problema

L'implementazione precedente generava falsi negativi. Se uno sviluppatore istanziava un modello specificando un parametro non strutturale (es. `random_state=42`, `n_jobs=-1`) ma ometteva parametri fondamentali per la stabilità del modello (es. `n_estimators` per un `RandomForest` o `n_clusters` per `KMeans`), il sistema non segnalava alcun rischio. Questo lasciava il codice esposto a "Silent Failure" in caso di modifica dei default nelle librerie sottostanti per quei parametri critici.

2.4.3 Soluzione proposta

Il sistema verrà esteso introducendo una configurazione esplicita degli iperparametri critici per ciascun modello supportato (es. estendendo `models.csv`). L'analisi statica utilizzerà questa mappatura per verificare che, nell'istanziatura di un modello, siano presenti specificamente le chiavi di configurazione ritenute critiche. Se tali chiavi sono assenti tra gli argomenti passati, il sistema segnalerà lo smell, indipendentemente dalla presenza di altri parametri opzionali.

2.5 CR5: Nuova Metrica di “Densità degli Smell”

Tipologia di manutenzione: Evolutiva.

2.5.1 Descrizione

Si propone l'introduzione di una metrica statistica che metta in relazione il numero di errori rilevati con la dimensione del file analizzato, offrendo una nuova prospettiva analitica sui dati raccolti.

2.5.2 Il Problema

Attualmente, il sistema riporta il numero assoluto di problemi. Tuttavia, 10 errori in un file di 100 righe sono molto più gravi di 10 errori in un file di 10.000 righe. Senza questo contesto, è difficile per i manager e gli sviluppatori capire quali file richiedono un intervento prioritario.

2.5.3 Soluzione proposta

Il sistema calcolerà un indice di “Densità” (Numero di problemi / Dimensioni del codice). Questo valore permetterà di classificare i file e i progetti in base alla loro qualità reale, fornendo una visione immediata delle aree più critiche (es. densità Bassa, Media, Alta) nei report finali.

2.6 Riepilogo Stato Modifiche

ID	Titolo	Tipo	Priorità	Stato
CR1	Stabilità Ambiente Docker	Correttiva	Alta	In Analisi
CR2	Progress Bar CLI	Perfettiva	Media	In Analisi
CR3	Smell: Randomness Uncontrolled	Evolutiva	Alta	In Analisi
CR4	Smell: Hyperparameter not explicitly set	Perfettiva	Media	In Analisi
CR5	Metrica: Densità Smell	Evolutiva	Alta	In Analisi

Tabella 2.1: Tabella riassuntiva delle Change Requests